



Welcome to this introductory course. I'm your guide, Pablo Farias. We will be diving into the practical ways you can utilize large language models for your personal or professional endeavors. This could be answering queries, finding relations between documents, or creating summaries across a wide range of data sources.

A Problem and A Solution

In practice, you may find yourself with hundreds of documents that need to be processed. However, large language models have a limited context window, meaning they can only process a certain amount of words. So, how can you work around this limitation?

This course offers a solution – a Python library called LlamaIndex. This tool aids in indexing documents, facilitating the efficient running of queries and providing answers using large language models. In technical terms, it facilitates a process called Retrieval Augmented Generation (RAG).

Understanding Retrieval Augmented Generation (RAG)

The concept of RAG hinges on indexing documents. When a query is issued, the index helps scan through documents to find relevant information. This information, alongside your query, is then processed by the large language model to give you the answer you want.

LlamaIndex: A Closer Look

LlamaIndex is an open-source library for Python, developed by Jerry Lu alongside a productive team. It's essentially a data framework designed for applications that utilize large language models. It's highly esteemed for its versatility in supporting different types of data sources and file formats, making it an excellent tool for small to enterprise-scale projects.

Course Goals

Our ultimate goal in this mini-course is to help you create applications that employ large language models to query your data using LlamaIndex. Following the course, you should be able to:

- Understand the primary components of a RAG system.
- Utilize documents metadata for more effective data retrieval.
- Assemble the specifics required for summarization in RAG, through LlamaIndex.

Basic Requirements

Assuming you're new to the large language model/AI sphere, we have tailored this course with beginners in mind. A foundational to intermediate knowledge of Python is, however, required.

Additionally, you will be required to obtain an API key from OpenAI and install specific packages. We will cover these aspects during the course.

We are excited to have you on board and looking forward to seeing the applications you will create with the new skills you acquire. So, let's get started!

In today's age of information, having to manually search through thousands of documents for particular information can be a time-consuming process. Even with the use of keyword searching, one may still miss out on pertinent information as it may be hidden under different terminologies. Instead, an intelligent approach that uses language understanding can play a pivotal role in document retrieval. When we add this with the power of augmented generations, we get a concept known as Retrieval-Augmented Generation (RAG).

What is Retrieval-Augmented Generation (RAG)?

RAG is a methodology that uses a large language model to understand a user's query and uses indexed documents to provide an accurate response. This model's operation can be divided into two major steps, first being document indexing and the second being document retrieval.

Document Indexing

Once, you have fed all your documents in the system, it will first create an index of all the documents. This is a crucial step as it catalogs all the information which enables it to find the relevant segments from the documents later, based on the user's query.

Document Retrieval

With the cataloging done, now the user can enter a query. A query could be a summary of a document, a specific question about something in those documents, or asking about how different documents relate to each other. The index, already made, would be used to find the relevant documents. The relevant documents or part of them get passed onto the large language model with the user's query. However, as different types of queries require a different type of handling, the system will have a query processing module. Using the index, the query processing model grabs all the relevant documents and those are then passed to the large language model.

Tools for RAG

One of the well-known Python libraries to implement a RAG model is "LlamaIndex". It allows you to talk to documents using a few lines of code. It not only supports files but also supports web data. It even has got connectors which can help you connect with tools like Notion or Trello or Google Calendar and use them as a source for documents.

To summarise, a RAG system is an efficient and time-saving approach to retrieve precise answers from a pile of documents. And with the use of tools like LlamaIndex, building your own RAG applications would be a breeze.

In the following guide, we will dive deep into how to use LlamaIndex to build own RAG applications. So stay tuned and get ready to explore.

The focus of this article will be on the components needed to run your applications. The primary tools we'll be needing are Python and a code editor. In this example, we'll utilize Visual Studio Code, but any code editor will do. We'll also discuss the necessary packages for success and how to obtain an API key from OpenAI to use its large language model service. This API key will be saved as an environmental variable for added security.

Packages

Several packages will have to be installed to operate the projects in this course. These are namely:

- LlamaIndex
- pypdf
- DOCX2TXT
- NLTK
- OpenAI
- Pathlib

LlamaIndex, a package being frequently updated, might lead to a package-missing error. This can be fixed by typing "deep install" followed by the name of the needed package.

API Key

To utilize OpenAI large language models, an API key is necessary. This can be obtained by signing up at platform.openai.com. Keep in mind that a credit card is needed at the time of registration. After registration, the API key can be obtained under 'View API Keys' section.

Note: Always make sure to keep your API key secure. If anyone gains access to it, they can use OpenAI's services at your cost. It's also crucial to not include them in the codes if you're planning to upload that code on platforms like GitHub.

Environmental Variable

The API key will be saved as an environmental variable for added security. What is an environmental variable? It's a variable that is stored in your computer session, in your development environment, so you can obtain it from the code without having to type the key in the code. Its name should be 'OpenAI_API_key'.

Setting Environmental Variable

On Windows:

1. Type 'Environmental Variables' in the Start menu.
2. Click on 'Edit Environmental Variables for your account' in the control panel.
3. Click on 'Environmental Variables' and click 'new' in the 'System Variables' section.
4. Name the variable 'OPENAI_API_KEY' and enter the obtained key value.

On Mac:

1. Open the '.zshrc' file, usually located in the home folder.
2. At the end of the file, add: `export OPENAI_API_KEY="Your API Key Value"`
3. Reload terminal.

Please note that for older versions of Mac OS or Linux, instead of '.zshrc', you'll need to open a file named 'bashrc'.



Now, with all required packages installed, an acquired OpenAI key, and 'OpenAI_API_key' environmental variable set up, we're ready to start building our first application with LlamaIndex.

Python Packages

This is the list of packages that need to be installed in your development environment:

```
pip install llama-index
pip install llama-index-core
pip install llama-index-llms-openai
pip install pypdf
pip install docx2txt
pip install nltk
pip install openai
pip install pathlib
```

These can be installed directly in your operating system (by using the pip command as per the snippet above) and that should be enough in order to be able to follow along.

An optional alternative to setup these packages is to use a **virtual environment**. This option is suitable for users who are more familiar with Python and with how to run terminal commands.

What is a Virtual Environment?

A virtual environment is a tool that helps you keep the dependencies used by different Python projects separate. Think of it as a unique playground where your project lives, isolated from other projects, avoiding any conflicts in package versions and settings. This way, if one project needs an older version of a library and another project needs a newer version, they can coexist peacefully on the same system. Using virtual environments allows for cleaner, more controlled, and reproducible development, making it easier to manage projects and their dependencies. It's a handy tool that ensures "it works on my machine" truly means "it works on my machine."

Setting up a Virtual Environment

Steps to setup your virtual environment:

1) Create a folder for your project. Place the "requirements.txt" file that we provide under Course Files in this new folder.

2) Open a terminal (on Windows, use the Visual Studio Code terminal or Power Shell), navigate into the new folder, type the following to create a virtual environment:

```
# navigate into your new folder using the cd command
python -m venv env
```

Note on older Mac or Linux you might need to type "python3" instead of "python"

3) Activate the virtual environment:

```
# Windows (using Power Shell or the terminal in Visual Studio Code)
.\env\Scripts\activate
```

```
# Mac or Linux  
source env/bin/activate
```

4) Install the packages in requirements.txt:

```
pip install -r requirements.txt
```

5) That's it! Type: **pip freeze** and you should see the list of packages that are installed in the environment. The list will be longer than requirements.txt, as the dependencies of those packages should be installed as well.

NOTE - If you're having errors when running your code in your virtual environment after having followed all the steps above, try running this extra command in your terminal:

```
pip install setuptools
```

You will need to activate the development environment each time you open the terminal in order to use it. For more information visit the official [documentation](https://docs.llama-index.ai/) and [our tutorial](https://academy.zenva.com).



Since the release of this course, some updates were made to LlamaIndex that impact the code seen in the lessons. For that reason, some lines of code needed to be updated for the source code to run correctly with Llama's latest install version. You can already find these corrections within the downloadable source files and noted with each lesson. However, you can find a summary of the key changes mentioned below.

What changed?

The difference for most of the lessons is in the **import** that changed from *llama_index* to ***llama_index.core*** as seen below:

Previous versions	Updated version
<code>from llama_index import VectorStoreIndex</code>	<code>from llama_index import VectorStoreIndex</code>

Apart from that, the *ServiceContext* container was deprecated and has been replaced by a bundle called ***Settings*** in lesson 11:

Previous versions	Updated version
<code>from llama_index import ServiceContext, set_global_service_context# configure service context service_context = ServiceContext.from_defaults(llm=llm) set_global_service_context(service_context)</code>	<code>from llama_index.core import Settings# configure service context Settings.llm = llm</code>

This is all! Enjoy your learning!

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, we will build a chatbot that allows us to interact with our documents using LlamaIndex, a powerful tool for creating conversational AI systems. This chatbot will be capable of answering questions based on the content of the documents. We will accomplish this in just five lines of code.

Setting Up

First, create a new folder and inside it, create a new file called ChatExample.py. Also, create a subfolder called Documents. In this subfolder, you should have three files provided as examples. These files are a 19-page policy document, a set of onboarding slides in PDF format, and a full export of a knowledge base portal in CSV format. These files will serve as the 'knowledge' of our chatbot.

Importing the Necessary Modules

The first step in our code is to import the necessary modules from LlamaIndex. We will need the VectorStoreIndex and SimpleDirectoryReader classes.

The following code has been updated and differs from the video:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
```

Loading the Documents

Next, we load the documents from our drive using the SimpleDirectoryReader class. This class will read all the files in the specified folder and load them as documents using LlamaIndex.

```
documents = SimpleDirectoryReader('documents').load_data()
```

Creating the Index

Once the documents are loaded, we create an index using the VectorStoreIndex class. This index will serve as the 'knowledge base' for our chatbot, allowing it to answer questions based on the content of the documents.

```
index = VectorStoreIndex.from_documents(documents)
```

Creating the Chat Engine

After the index is created, we create a chat engine that will use this index to answer questions. We use the as_chat_engine method of the VectorStoreIndex class to accomplish this. We also set the verbose parameter to True to log more information in the console about what's going on.

```
chat_engine = index.as_chat_engine(verbose=True, chat_mode='react')
```




Finally, we start the chat by calling the `chat_repl` method of the chat engine. This will open an interactive console for us to talk to the chatbot.

```
chat_engine.chat_repl()
```

Testing the Chatbot

To test the chatbot, run the `ChatExample.py` script. Once the script is executed, you can start asking questions to the chatbot in the console. Note that the chatbot's responses will be based on the content of the documents loaded earlier.

If you find that the chatbot is not answering questions based on the documents, you may need to set the `chat_mode` parameter to 'react' when creating the chat engine. This will constrain the chatbot to only talk about what's in the documents.

Conclusion

In this lesson, we have built a simple yet powerful chatbot using LlamaIndex. This chatbot can answer questions based on the content of a set of documents. This is a great way to provide interactive access to large volumes of text, such as policy documents or knowledge bases. In the next lesson, we will look at how to save the index for later use, reducing the time it takes to load the documents and create the index each time.

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, we will learn how to save our index to disk and load it from a different file, eliminating the need to recreate the index every time we run our script. This process is useful for managing larger datasets and improving efficiency in your programs.

Setting up the index

First, let's create a new file named 'Save_index.py'. The first three lines of code will be the same as in our previous 'Chat_example.py' file. This code will import necessary functions, read and load data from a specified directory, and construct an index from the loaded documents.

The following code has been updated and differs from the video:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents
documents = SimpleDirectoryReader('documents').load_data()

# create index
index = VectorStoreIndex.from_documents(documents)
```

Saving the index

Once we have created our index, we need to save it. We can do this by typing the following command:

```
index.storage_context.persist()
```

When you execute this code, it will create a folder called 'storage'. This folder contains a few different JSON files that store all the information in your index. Each document has its unique identifiers, and all the content of your documents is separated into different documents. Thus, the information is stored as a large text file with all the data.

Loading the index

To load the index, we will create a new file named 'Query_index.py'. In this file, we will import two objects from the llama_index module: 'StorageContext' and 'load_index_from_storage'. These objects are essential for loading the index and using it.

The following code has been updated and differs from the video:

```
from llama_index.core import StorageContext, load_index_from_storage
```

First, we rebuild the StorageContext by loading the files that were saved. We do this by specifying the directory where our index was saved. In our case, it was saved in a directory called 'storage' located in the same folder where we are.

```
# rebuilding storage context
storage_context = StorageContext.from_defaults(persist_dir='./storage')
```

Next, we load the index from the storage context. This process differs from how we did it before. Previously, we loaded the index from the documents where we loaded the documents and then created the index from them. In this case, we load a storage context and then create the index from that.

```
# load index
index = load_index_from_storage(storage_context)
```

Creating a query engine

Once we have the index, we can create a query engine. The query engine allows us to ask specific questions and print the response. In our example, we are asking for the link to the article with instructions on how to create a new teacher account.

```
query_engine = index.as_query_engine(verbose=True)

response = query_engine.query('give me the link to the article with instructions on how to create a new teacher account')

print(response)
```

When you run this script, it will return the requested link. This script is not interactive; it only runs a single query. However, you can easily modify it to run more queries as needed.

Summary

In this lesson, we have learned how to save our index to disk and load it from a different file. This process allows us to manage larger datasets efficiently and improve the performance of our scripts. Remember, all the data is stored in the 'storage' directory, and we can directly load the index from this directory.



In this article, we're going to delve into the concept of embeddings, discussing what they are and why they're so useful in the context of searching through large amounts of text data. We'll also touch on how these embeddings are stored for efficient retrieval.

What are Embeddings?

Consider a scenario where you have hundreds of text documents, and you're interested in finding information on 'fitness routines'. A traditional word matching search would simply look for occurrences of 'fitness' in the documents but would ignore semantically related terms such as 'exercise plan' or 'workout routine'. A simple word search is not efficient or accurate, and writing a list of every possible related word isn't feasible or scalable. This is where embeddings can become incredibly helpful.

Embeddings are essentially a way to represent words as numerical vectors. This technique allows us to search documents in a more meaningful and intelligent manner. Words with similar meanings like 'exercise', 'workout', and 'practice' will have similar numerical representation and thus, will be closer in a multi-dimensional space, making the search and retrieval more semantically accurate.

Why are Embeddings Important?

Embeddings are effective for several reasons:

- They allow for context-aware searches and applications, as they're not just matching strings.
- Embeddings can handle synonyms and produce intelligent recommendations.
- They can understand and distinguish the context in words, such as 'Apple' the company vs 'apple' the fruit or 'workout' vs 'exercise'.

These qualities make embeddings widely popular in fields like natural language processing and machine learning, particularly in creating large language models.

Practical Applications

One practical application of embeddings is the capability to chat or perform queries with your documents. The documents are transformed into embeddings and stored in a vector index. Retrieval augmented generation tools, such as LlamaIndex, are then used to process user queries.

Storing Embeddings

When it comes to storing embeddings, if the scale of implementation is small, storing them as a JSON file will suffice. However, when deploying on an enterprise scale, an efficient storage solution such as a vector database is needed. Vector databases like Chroma, Pinecone are designed specifically for storing embeddings and running queries efficiently.

In Summary

Traditional word search methods have their limitations—they don't pick up on synonyms or semantically close terms, which can lead to incomplete or inaccurate results. Embeddings offer a robust solution to these issues, enabling intelligent, context-aware searches and applications. With embeddings, it becomes possible to interact with your documents, almost as if you're chatting with them. Efficient storage and retrieval of embeddings are critical for implementing this technology at a larger scale.

In the end, the power of embeddings lies in their capability to bring semantic understanding into text searches, making them a crucial tool in current data-driven industries.



Note that in the next lesson, the instructor uses the *ServiceContext* container in the video lesson. However, it was deprecated and has been replaced by a bundle called **Settings**. So the changes in the code are as follows:

Outdated code

Updated code

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, we will learn how to change the default models used by the LlamaIndex. At the time of this recording, the default models are GPT 3.5 Turbo and text embedding data. However, you can actually change the default models to any of the main large language models that exist out there. Let's go through the steps to specify a different model, particularly GPT-4.

Before we begin, copy the chat example code we used in a previous lesson, as we'll be adding on top of that to set up our custom LLM.

1. Importing Additional Objects

The first step is to add some additional imports that will be used to specify a different model. As we're going to use the GPT 4, we'll need to import it from the llama_index module.

Note that *ServiceContext* is now *deprecated* and has been replaced by the *Settings* bundle:

The following code has been updated and differs from the video:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.llms.openai import OpenAI
import openai
```

2. Defining the Large Language Model

Next, we will define the large language model (LLM). For this example, we will use the GPT-4 model.

```
llm = OpenAI(model="gpt-4", temperature=0, max_tokens=1000)
```

In the above code, the OpenAI object is instantiated with the model set to "gpt-4", temperature set to 0, and max_tokens set to 1000. The configuration is saved in the variable llm.

3. Setting up the LLM

As ServiceContext and set_global_service_context are deprecated, we're using Settings to set our LLM variable as the global default for all actions needed:

The following code has been updated and differs from the video:

```
Settings.llm = llm
```

4. Checking the Execution of the Correct Model

To check that the correct model is executing, and also to get more visibility in regards to the API calls, we will import the OpenAI package and set the log level to debug.

```
import openai
```

```
openai.log = 'debug'
```

Note: To use this feature, you need to have the OpenAI package installed, which can be installed by typing `pip install openai` in your terminal.

5. Executing the Script

Now that we have set up our script, we can execute it and ask any random question. For example, “how do I log in?”. If the script is working correctly, you will see a debug message indicating that a request has been made to OpenAI with the specified model, temperature, and maximum tokens.

In conclusion, we now understand that there are default models that the LlamaIndex library uses. However, these default models can be changed, and there is wide support for different large language models that are available in the market. This lesson has shown us the way to set up these models.

The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, we'll explore how to add extra functionality to your knowledge management system by enabling search by file name and defining metadata such as the category of the document, the project it belongs to, or tags. We'll cover how to load a single file instead of an entire directory, what happens if we load multiple directories, how to combine them, and how to add metadata to your documents.

Once again before starting, copy the chat example code we created in an earlier lesson, as we'll once again be adding to that.

Note that the *import* now is as follows:

The following code has been updated and differs from the video:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader
```

Loading a Single File

First, let's learn how to load a single file. In the SimpleDirectoryReader where we load documents, instead of specifying the name of the folder, we are going to pass a parameter called `input_files`. This parameter should be a list of files. If you only want to load one file, you can just add the location of that file as a string. Here's how to do it:

```
docs1 = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge base .csv']).load_data()
```

The `input_files` parameter takes a list of file paths. The path starts from the current folder (indicated by the dot) and then proceeds to the folder and file name.

Loading Multiple Directories

Now, let's learn how to work with multiple folders. We can load the second folder the same way we did before:

```
docs2 = SimpleDirectoryReader('docs2').load_data()
```

Since each of the loaded documents is a list, we can combine them by simply adding the lists together:

```
all_docs = docs1 + docs2
```

Adding Metadata



Now let's add some metadata to our documents. Metadata can be anything you want, such as categories, topics, projects, or tags. Here's an example of how to add a category to your documents:

```
docs1[0].metadata['category'] = 'customer support'

for doc in docs2:
    doc.metadata['category'] = 'marketing'
```

The first line assigns the category 'customer support' to the first document in docs1. The for loop assigns the category 'marketing' to all documents in docs2.

Creating and Saving the Index

Once we've loaded and categorized our documents, we can create an index from these documents and save it:

```
index = VectorStoreIndex.from_documents(all_docs)
index.storage_context.persist()
```

Querying the Index

Finally, we can query the index to retrieve documents based on their metadata. For example, we can ask for a summary of all the customer support content

```
query_engine = index.as_query_engine(verbose=True)
response = query_engine.query('summarize our customer support content')
print(response)
```

Keep in mind that while this works well most of the time, it's not always perfect due to the complexities of large language models and the embedding system. It's possible that it might not always return the correct documents or all the documents in a certain category.

In conclusion, by using metadata, we can significantly improve our ability to query our documents. We can now not only talk to the AI about what's in the documents but also about what categories they're in, their file names, and more. This is extremely helpful when you're trying to retrieve specific information and is a common feature in most knowledge management systems.



The code in the video for this particular lesson has been updated, please see the lesson notes below for the corrected code.

In this lesson, we will discuss the concept of summarization in the context of retrieval augmented generation and the Llama Index Framework. Summarization is a crucial aspect to consider when we are querying information or searching for answers. However, it functions somewhat differently, and there are specific aspects to remember. To explain this, we will use a basic example, and you can find the contents of this file in the 'summarization starter'.

Understanding Summarization

Firstly, we will load a single file and assign it a category. Then, we generate our Vector Store Index, save it, and run a query. The query is to summarize our customer support content. Let's look at the code snippet for this:

The following code has been updated and differs from the video:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents
docs = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge base.
csv']).load_data()
docs[0].metadata['category'] = 'customer support'

# create index
index = VectorStoreIndex.from_documents(docs)

# save index
index.storage_context.persist()

# query
query_engine = index.as_query_engine(verbose=True)
response = query_engine.query('summarize our customer support content')
print(response)
```

After executing this code, we get a summary of the customer support content. However, the summary only includes a small part of what the document covers. This is because the Vector Store Index, by default, uses only two nodes in retrieval. So, when we run a query, it only retrieves two chunks of text from the document. If we want to see which parts of the document have been used for the summary, we can use the following command:

```
print(response.get_formatted_sources())
```

This command will print out a small preview of the nodes that have been used for the summary. If you want to see the full content of each node, you can use the following command:

```
print(response.source_nodes)
```



Improving Summarization

There are several ways to improve the quality of your summarization. However, the most effective method will depend on the specifics of your project. Here are a few general strategies:

- **Increase the number of nodes retrieved:** You can increase the number of nodes retrieved in the Vector Index. This will result in a more comprehensive summary. However, increasing the number of nodes will also increase the cost and the time it takes to get a response. Here's how you can increase the number of nodes:

```
query_engine = index.as_query_engine(verbose=True, similarity_top_k = 6)
```

- **Change the response mode:** The query engine has a default response mode called 'compact', which concatenates chunks of nodes and passes them into the large language model. However, you can change the response mode to 'tree_summarize', which is more suited for summarization. Here's how you can change the response mode:

```
query_engine = index.as_query_engine(verbose=True, similarity_top_k = 6, response_mode='tree_summarize')
```

- **Use a summary index:** If you want to go through all the nodes in the document, you can use a Summary Index, which is optimized for summaries. However, this can significantly increase the time it takes to process the document. Here's how you can use a Summary Index:

The following code has been updated and differs from the video:

```
from llama_index.core import SummaryIndex
```

```
# create index  
index = SummaryIndex.from_documents(docs)
```

In conclusion, summarization in the Llama Index Framework involves a careful balance between the quality of the summary, the cost, and the time it takes to generate the summary. By adjusting the number of nodes, the response mode, and the type of index, you can optimize the summarization process for your specific project.



Congratulations on completing this course on how to use large language models for querying documents via the LlamaIndex! The course started with a broad base of documents and guided you through how to query these documents, with a particular focus on how to use large language models to understand these documents and execute complex queries.

Using LlamaIndex

Throughout the course, you learned how to use LlamaIndex to build retrieval augmented generation (RAG) systems that index documents, communicate with large language models, and provide the answers for your queries. RAG systems operate by constructing an index of older documents, allowing them to query these documents and identify the relevant chunks or nodes, which can be introduced to a large language model to facilitate answering.

Why LlamaIndex?

LlamaIndex was the chosen tool for this course due to its open-source availability and robust functionality as a Python library. LlamaIndex serves as a powerful data framework for LLM applications, which can be utilized to create a RAG system in a few lines of code. It supports a range of data sources with a growing ecosystem of connectors, making it suitable for both small projects and enterprise-level applications.

What was achieved?

By finishing this course, you have learned:

1. How to build applications that leverage large language models to query your documents using LlamaIndex.
2. The various components of a RAG system.
3. Understanding document metadata and specifics of summarization.

What next?

Zenva is a learning platform that offers courses in a multitude of domains embracing coding, data AI, and game creation. With hundreds of courses spanning a wide range of topics, your next course to continue your learning journey awaits in its catalog. Keep up the spirit of always learning and looking forward to hearing about your amazing AI applications!

Thank you for choosing this course and happy learning!

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Chat_example.py

Found in Project root folder

This code loads a data set of “documents” using the SimpleDirectoryReader class from the llama_index module. It then creates an index and transforms it into a chat engine. Finally, it runs a chat bot in reactive mode that gives verbose output.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents
documents = SimpleDirectoryReader('documents').load_data()

# create index
index = VectorStoreIndex.from_documents(documents)

# create chatbot
chat_engine = index.as_chat_engine(verbose=True, chat_mode='react')
chat_engine.chat_repl()
```

Save_index.py

Found in Project root folder

This code imports necessary functions, reads and loads data from a specified directory, constructs an index from the loaded documents, and then saves that index. It’s designed to create an organized structure(Index) for faster search and retrieval of data from the documents directory.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents
documents = SimpleDirectoryReader('documents').load_data()

# create index
index = VectorStoreIndex.from_documents(documents)

# save index
index.storage_context.persist()
```

Query_index.py

Found in Project root folder

This code is responsible for initializing a StorageContext from a pre-defined directory, loading an

index from storage, and setting up a query engine. It then queries for a specific piece of information and prints the response.

```
from llama_index.core import StorageContext, load_index_from_storage

# rebuilding storage context
storage_context = StorageContext.from_defaults(persist_dir='./storage')

# load index
index = load_index_from_storage(storage_context)

query_engine = index.as_query_engine(verbose=True)

response = query_engine.query('give me the link to the article with instructions on how to create a new teacher account')

print(response)
```

Custom_llm.py

Found in Project root folder

This code sets up a chatbot that uses the OpenAI GPT-4 model for natural language understanding, using the Llama-based indexing system. It reads in a set of documents, uses them to create a knowledge index, and then allows for interactive, real-time chat based on this index.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Settings
from llama_index.llms.openai import OpenAI
import openai

openai.log = 'debug'

# define LLM
llm = OpenAI(model="gpt-4", temperature=0, max_tokens=1000)

# configure service context
Settings.llm = llm

# load documents
documents = SimpleDirectoryReader('documents').load_data()

# create index
index = VectorStoreIndex.from_documents(documents)

# create chatbot
chat_engine = index.as_chat_engine(verbose=True, chat_mode='react')
chat_engine.chat_repl()
```

Metadata.py

Found in Project root folder

This code imports classes from a llama_index module, loads and categorizes two types of documents for customer support and marketing respectively. It generates a combined index from these categorized documents and saves it. Finally, a chat engine is created using this index and a chat session is started.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents
docs1 = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge base
.csv']).load_data()
docs2 = SimpleDirectoryReader('docs2').load_data()

docs1[0].metadata['category'] = 'customer support'

for doc in docs2:
    doc.metadata['category'] = 'marketing'

all_docs = docs1 + docs2

# create index
index = VectorStoreIndex.from_documents(all_docs)

# save index
index.storage_context.persist()

# create chatbot
chat_engine = index.as_chat_engine(verbose=True, chat_mode='react')
chat_engine.chat_repl()
```

Summarization_starter.py

Found in Project root folder

This code loads documents from a specified file, assigns a category to them, and creates an index. The index is then saved before a query is made to summarize content related to customer support. Finally, the query response is printed out.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

# load documents
docs = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge base.
.csv']).load_data()
docs[0].metadata['category'] = 'customer support'

# create index
index = VectorStoreIndex.from_documents(docs)

# save index
index.storage_context.persist()

# query
query_engine = index.as_query_engine(verbose=True)
response = query_engine.query('summarize our customer support content')
```

```
print(response)
```

Summarization.py

Found in Project root folder

This code reads a set of documents from a directory and categorizes them as 'customer support'. It then creates an index for the documents and stores it. The last part of the code queries the indexed documents for customer support content and outputs a summary response, additionally printing relevant sources.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, SummaryIndex

# load documents
docs = SimpleDirectoryReader(input_files=['./docs1/example - exported knowledge base.
csv']).load_data()
docs[0].metadata['category'] = 'customer support'

# create index
index = VectorStoreIndex.from_documents(docs)

# save index
index.storage_context.persist()

# query
query_engine = index.as_query_engine(verbose=True, similarity_top_k = 6, response_mod
e='tree_summarize')
response = query_engine.query('summarize our customer support content')
print(response)

print(response.get_formatted_sources())

#print(response.source_nodes)
```